

AI Vehicle Damage & Repair Cost Estimator

Project Problem Statement, Technical Approach & Implementation Blueprint

Core idea

Upload a damaged vehicle image and receive an AI-assisted assessment of visible damage, affected parts, severity, recommended repair/replacement action, estimated repair cost, and compatible part-price comparisons.

Target outcome

Build a credible end-to-end AI product - not just an image classifier - combining computer vision, estimation logic, structured vehicle/parts data, and a production-style web interface.

Recommended product name: AutoInspect AI

Tagline: From accident image to repair estimate.

Important scope note: The system is an image-based preliminary assessment tool. It must not claim to detect hidden mechanical/structural damage or guarantee that a part is safe to repair or replace.

1. Problem Statement

After a collision or visible vehicle damage, car owners often do not know exactly which exterior parts are affected, whether a component is more likely to require repair or replacement, or how much the repair may cost. The information is usually fragmented across body shops, part catalogs, online listings, and manual inspection. Create an AI-powered web platform that analyzes a user-provided vehicle image, identifies visible damaged regions and affected components, classifies the damage, estimates severity, recommends a likely repair/replacement action, estimates a cost range, and compares compatible part listings.

The core user question is:

"I uploaded a photo of my damaged car. What is visibly damaged, what will probably need repair or replacement, how much might it cost, and where can I find the relevant part at a good price?"

The system should be designed as a decision-support tool rather than a substitute for a qualified vehicle inspection.

2. Objectives

- Detect visible vehicle damage in uploaded photographs using computer vision.
- Localize damaged regions with bounding boxes or segmentation masks.
- Identify affected vehicle components such as bumper, fender, door, hood, headlight, taillight, mirror, grille, and windshield.
- Classify damage type such as scratch, dent, crack, broken, or severe deformation, where supported by the data.
- Estimate visible damage severity as Minor / Moderate / Severe using measurable visual signals and labeled examples.
- Recommend repair vs. replacement as a probabilistic or rules-based recommendation, not an absolute diagnosis.
- Estimate repair cost as a range using part cost, labor assumptions, painting, and other configurable factors.
- Match compatible parts using vehicle make/model/year/variant and part identifiers.
- Compare listed prices across approved/available sources or an internal structured marketplace dataset.
- Generate a clear assessment report containing the image, detected damage, confidence, estimate, and caveats.

3. MVP Scope - What We Will Actually Build

To keep the project technically credible and finishable, the first production-quality MVP should cover a constrained set of exterior parts rather than every possible vehicle component.

Category	Initial MVP scope
Vehicle parts	Front bumper, rear bumper, hood, fender, door, headlight, taillight, side mirror, grille, windshield
Damage classes	Scratch, dent, crack, broken, deformation/severe damage
Severity	Minor, Moderate, Severe

Category	Initial MVP scope
Input	1-3 user-uploaded exterior vehicle photos
Output	Damage visualization, affected part, damage class, confidence, severity, recommended action, cost
Parts pricing	Structured listings with seller, part number, price, URL, availability, and timestamp
Report	Web dashboard + downloadable PDF assessment

Out of scope for the first version: hidden frame damage, engine/internal mechanical diagnosis, airbag deployment diagnosis, legal/insurance decisions, guaranteed repairability, guaranteed cheapest internet price, and autonomous safety certification.

4. High-Level System Architecture

Pipeline:

User Image -> Preprocessing -> Vehicle/Part Detection -> Damage Detection/Segmentation -> Damage Classification -> Severity Estimation -> Repair/Replacement Recommendation -> Vehicle/Parts Database -> Cost Estimation -> Price Comparison -> Report

Layer	Responsibility	Suggested technology
Frontend	Image upload, vehicle selection, visualization, Results, Tailwind CSS, report	React, Tailwind CSS
API / Backend	Authentication, inference orchestration, database access, pricing, report generation	FastAPI, Python
Computer vision	Part/damage detection and segmentation/classification	PyTorch + a YOLO-family detector/segmenter or comparable C
Image processing	Resize, normalization, augmentation, visualization	OpenCV
Database	Vehicles, parts, OEM numbers, listings, prices, PostgreSQL or Firebase	PostgreSQL or Firebase
Cost engine	Part + labor + paint + configurable assumptions	Python rules/model
Report	Assessment PDF generation	ReportLab or backend PDF library
Deployment	Hosted web app and API	Vercel + Render/Railway/AWS/etc.

5. Dataset Strategy

Do not search for one dataset that magically solves the whole product. Treat the dataset problem as multiple connected tasks.

Dataset need	What it should contain	Use in project
Damage dataset	Damaged vehicle images with damage localization/region/segmentation and damage labels	Train/infer damage detector/s
Vehicle-part dataset	Part-level labels for exterior components	Teach the system which component a damaged region
Severity data	Images/examples with consistent severity labels	Train or calibrate severity estimation
Price/catalog data	Vehicle make/model/year/variant, part number, seller, price, URL and cost/price comparison	Complete URL and cost/price comparison
Own validation set	Realistic unseen images captured/collected with permission	Final generalization evaluation

CARDD: Use the CARDD dataset as a candidate primary dataset for the computer-vision damage component. Before training, inspect the exact release annotations, damage classes, image counts, annotation format, and license. CARDD should be treated as a damage-localization/segmentation input to the project, not assumed to contain every vehicle-part, severity, or pricing label required by the complete product.

Dataset rule: Check the license/terms of every source before redistribution or commercial use. Keep dataset source and license information in the repository.

6. Machine Learning Approach

Recommended initial design: a modular multi-stage vision pipeline. This is easier to evaluate and explain than a single monolithic classifier.

Stage 1 - Vehicle/Part Localization

- Detect or segment relevant exterior components.
- Output class, bounding box/mask, and confidence.
- If the chosen damage dataset already provides useful part/damage localization, reuse that signal where appropriate instead of duplicating work.

Stage 2 - Damage Detection / Segmentation

- Detect the visible damaged region.
- Prefer segmentation when masks are available because a damaged-area ratio can support severity estimation.
- Preserve the original image and overlay predictions for the UI.

Stage 3 - Damage Type

- Classify the localized region as scratch / dent / crack / broken / deformation where the dataset supports these classes.
- Use confidence thresholds and return 'uncertain' when evidence is weak rather than forcing a class.

Stage 4 - Severity Estimation

Build a separate severity component using labeled examples plus measurable visual features such as damaged-area ratio, region size, damage type, edge complexity, and model confidence. Start with Minor / Moderate / Severe. Do not invent labels that the dataset cannot justify.

Stage 5 - Repair / Replacement Recommendation

Use a transparent rules/model layer that combines part type, damage type, severity, and visual evidence. Example: severe cracked headlight -> replacement likely; minor bumper dent -> repair likely. Every recommendation should retain an explanation and confidence.

7. Cost Estimation & Parts Engine

Separate observed market/listing price from estimated repair cost.

Component	Example calculation
Replacement part	Selected compatible part/listing price
Labor	Configurable labor rate x estimated labor time
Painting	Configurable paint/finish estimate where relevant
Miscellaneous	Optional configurable shop/material cost
Total estimate	Part + labor + paint + miscellaneous

Return a range such as ₹18,000-₹23,000 instead of pretending that an image gives an exact final bill. Store the assumptions used for the estimate so the user can understand where the number came from.

For price comparison, initially use a structured catalog/listing database. Add live source integrations only where access and terms permit. The product should say "best price among indexed listings", not "cheapest price on the internet."

8. Core API / Data Contracts

Keep the AI output stable and machine-readable.

Example damage object

```
{ "part": "front_bumper", "damage_type": "crack", "severity": "severe", "confidence": 0.91, "recommended_action": "replace", "evidence": { "damage_area_ratio": 0.24 } }
```

Example analysis endpoint

POST /analyze -> image + vehicle_id -> vehicle details + detected damages + estimates + compatible parts

Example parts endpoint

GET /parts?vehicle_id=...&part=front_bumper -> compatible listings with seller, price, URL, availability, and last_checked

9. Evaluation Plan

Do not evaluate only by visual inspection of a few demo images. Keep a held-out test set that the model never sees during training.

Component	Metrics
Detection / segmentation	Precision, Recall, mAP; IoU/Dice where masks are used
Damage classification	Accuracy, Precision, Recall, F1, confusion matrix
Severity	Accuracy/F1 for classification or MAE/RMSE for a numeric score
Cost estimation	MAE, RMSE, percentage error; report median error too
End-to-end utility	Percentage of test images with correct major part + actionable assessment

Error analysis is mandatory. Document common failure cases: low light, heavy occlusion, unusual angles, multiple simultaneous damages, reflections, severe deformation, and unsupported vehicle variants.

10. Website Experience

Screen	Required contents
Home	Project explanation + upload CTA
Upload	Vehicle make/model/year/variant + one or more images
Analysis	Original image + bounding boxes/masks + confidence
Damage summary	Detected parts, type, severity, recommended action
Cost estimate	Part cost, labor, painting, total range, assumptions

Screen	Required contents
Part comparison	Compatible part number, seller, price, availability, link
Report	Downloadable assessment PDF

Recommended visual feature: an interactive image overlay. Clicking a damaged region should open its part name, damage type, severity, confidence, recommendation, and estimate.

11. Two-Person Work Split

Person A - AI/ML	Person B - Product/Engineering
Dataset audit & preprocessing	React UI
Annotation/label mapping	FastAPI endpoints
Detection/segmentation model	Database schema
Damage classification	Vehicle/parts catalog
Severity estimation	Price comparison service
Repair/replacement scoring	PDF report generation
Model evaluation/error analysis	Deployment, authentication, monitoring

Both people should be able to explain the full pipeline, model assumptions, dataset limitations, and evaluation results.

12. Implementation Roadmap

Phase	Deliverable	Exit condition
1. Specification	Classes, outputs, data schema, scope	Both developers agree on exact MVP
2. Dataset	Primary + supplementary datasets, labels, licenses	Training-ready dataset
3. Baseline CV	First detector/segmenter	Image -> visible damage result
4. Part + damage pipeline	Part + damage type	Component-level result
5. Severity	Severity estimator	Minor/Moderate/Severe with evaluation
6. Recommendation	Repair/replace engine	Traceable recommendation
7. Cost engine	Estimate range	Repeatable estimate from stored assumptions
8. Parts database	Compatibility + listings	Searchable part comparisons
9. Backend	FastAPI integration	End-to-end API works
10. Frontend	Dashboard + overlays	User can complete workflow
11. Report	PDF assessment	Downloadable report
12. Evaluation	Held-out test + metrics + errors	Results documented
13. Deployment	Hosted application	Public demo works

13. Master Prompt for Another AI Coding/Engineering Tool

Copy the following prompt into your coding AI. It is deliberately specific so the tool does not turn the project into a generic image classifier.

You are the lead AI engineer and full-stack architect for a project called AutoInspect AI. Goal: Build an end-to-end web platform that takes a user-uploaded vehicle image and a selected vehicle (make/model/year/variant), detects visible exterior damage, identifies affected parts, classifies damage type, estimates severity, recommends a likely repair/replacement action, estimates a repair-cost range, and shows compatible part listings and prices. Important product boundary: This is an AI-assisted preliminary assessment tool for visible damage. Do not claim to detect hidden structural/mechanical damage or guarantee repairability, safety, insurance decisions, exact prices, or the cheapest price on the entire internet. Initial supported parts: front bumper, rear bumper, hood, fender, door, headlight, taillight, side mirror, grille, windshield. Initial damage classes: scratch, dent, crack, broken, severe deformation. Severity: Minor, Moderate, Severe. Primary computer-vision strategy: use a modular pipeline rather than one monolithic predictor. Build or fine-tune a detector/segmenter for damaged regions; use vehicle-part labels where available; classify damage type from the localized region; estimate severity from labeled data plus measurable visual features; then pass structured outputs into a transparent repair/replacement recommendation layer. Dataset strategy: Treat CARDD as a candidate primary damage dataset, but inspect its actual annotations, classes, counts, and license before use. Do not assume CARDD contains every vehicle-part, severity, or price label required. Plan for supplementary part/severity data if needed. Keep dataset source and license metadata in the repository. Engineering stack: React + Tailwind for frontend, FastAPI + Python backend, PyTorch for ML, OpenCV for image processing, PostgreSQL or Firebase for structured data. Keep the AI model behind a clean inference service. Core data flow: image upload -> preprocessing -> damage/part detection -> damage type -> severity -> repair/replacement recommendation -> vehicle/parts lookup -> cost estimation -> price comparison -> report. Backend requirements: - POST /analyze - GET /vehicles - GET /parts - GET /listings - POST /reports - structured JSON responses - input validation - error handling - logging - configurable model path and thresholds - separate modules for model inference, business rules, pricing, and database access. Example analysis object: { "part": "front_bumper", "damage_type": "crack", "severity": "severe", "confidence": 0.91, "recommended_action": "replace", "evidence": { "damage_area_ratio": 0.24 } } Cost engine: estimate a range using part price + configurable labor + paint + miscellaneous cost. Store the assumptions used. Do not return fake precision. Parts engine: match by vehicle compatibility and part number where available. Return seller, price, URL, availability, and last_checked. Initially use a structured local catalog/listings dataset; create a provider interface so legal/approved live integrations can be added later. Frontend: build a professional dashboard with upload, vehicle selection, image overlay, clickable damage regions, damage summary, repair estimate, part comparison, and PDF report download. Evaluation: create a held-out test set. Report detection precision/recall/mAP, segmentation IoU/Dice when applicable, classification F1, severity performance, and cost-estimation MAE/RMSE/percentage error. Add an error-analysis section for low light, occlusion, angle, multiple damages, and unsupported vehicles. Implementation discipline: 1. Inspect the dataset before choosing the final model. 2. Build a baseline before optimizing. 3. Keep model, business logic, database, and UI separated. 4. Never fabricate metrics. 5. Never hard-code model predictions into the UI. 6. Log model version and thresholds. 7. Return uncertainty instead of forcing low-confidence predictions. 8. Write a README with architecture, setup, dataset licenses, training commands, evaluation, limitations, and demo instructions. 9. Build the smallest working end-to-end vertical slice first, then expand. First task: inspect the available dataset files and annotations, summarize their structure, map the labels to the project taxonomy, identify missing labels, and propose the exact training format. Do not start frontend development until the dataset schema and first baseline training plan are defined.

14. Immediate Next Steps

- Download and inspect CARDD and any candidate supplementary datasets.
- Record image count, annotation format, classes, sample images, and license/usage terms.
- Create a label-mapping document: dataset label -> project part/damage class.
- Build one notebook/script that loads an image and visualizes annotations.
- Train one baseline detector/segmenter before building advanced modules.
- Freeze the API output schema early so the AI and web teams can work independently.
- Create a small structured vehicle + parts + listings database for the first demo.
- Only after the baseline works, build the polished frontend and price-comparison layer.

15. Definition of Done

- A user can upload a vehicle image and select a supported vehicle.
- The system localizes at least one supported visible damage type on the held-out test set.
- The UI shows the damaged region on the original image.

- The system provides part, damage type, severity, confidence, and recommended action.
- The system produces a cost range with explicit assumptions.
- The system finds compatible part listings in its indexed catalog.
- The user can generate/download a report.
- The README contains real evaluation metrics, limitations, and dataset license information.
- The deployed demo completes the full workflow without manual intervention.

Final principle

Build the project around a real decision workflow rather than around a single model. The strongest story is "image -> evidence -> component assessment -> recommendation -> cost -> part options". That makes the project useful, technically defensible, and substantially stronger on a resume than a simple car-damage classifier.